

Summarized Technical Report: ANE-HX / MonRaF2 Spectrum Monitoring System

SDR Sensor Engineering Team

February 2026

1 System Design and Architecture

The **ANE-HX (MonRaF2)** system is an autonomous **Software-Defined Radio (SDR)** platform designed to capture, process, and report radioelectric spectrum data across a range of **1 MHz to 6 GHz**. The system architecture follows an **edge computing model**, where heavy signal processing is performed locally on the sensor to minimize network overhead.

1.1 High-Level Architecture

The system implements a **decoupled hybrid architecture** divided into two functional planes:

1. **Control Plane (Python 3):** Managed by the `orchestrator.py` module, which acts as the "cognitive unit" responsible for logic orchestration, state transitions, and cloud API communication.
2. **Data Plane (C99):** A high-performance **RF Engine** (`rf_app`) responsible for direct hardware interaction, IQ sample normalization, and intensive Digital Signal Processing (DSP) like **FFT** and **PFB** calculations.

1.2 Communication and Infrastructure

Inter-process communication (IPC) between Python and C is handled via **ZeroMQ (ZMQ)** sockets over the IPC protocol to ensure low latency and fault isolation. For remote management, the system utilizes **OTA-ANE v7**, an infrastructure employing **WireGuard VPN** for secure tunneling and **Port Knocking** to protect management ports from unauthorized access.

2 Technologies Employed

2.1 Hardware Stack

- **Raspberry Pi 5:** Serving as the primary processing unit with an **ARM Cortex-A76** CPU and 8 GB of RAM.

- **HackRF One:** The core SDR receiver providing up to **20 MHz** of instantaneous bandwidth.
- **SIM7600G-H Module:** Provides **4G/LTE** connectivity and **GNSS/GPS** for spatial-temporal synchronization.
- **Antenna System:** A switched array including **VHF/UHF (25 MHz-1 GHz)**, **TDT (470-698 MHz)**, and high-frequency omnidirectional antennas (**>2 GHz**).

2.2 Software and DSP Libraries

The system leverages **FFTW3** and **KissFFT** for frequency domain transformations. Audio streaming is achieved through a **WebRTC gateway** using **GStreamer** and the **Opus codec** to maintain low-latency distribution over unstable LTE links.

3 System Functionalities

3.1 Advanced Acquisition Strategies

To eliminate the **"DC Spike"** (center artifact) inherent in direct-conversion SDRs, the system uses two software strategies:

- **Offset & Crop (Real-time):** The Local Oscillator (LO) is tuned 1 MHz away from the target, and Gaussian noise is interpolated over the artifact before cropping the band.
- **Spectral Stitching (Campaign):** Two captures are taken at different offsets and surgically combined ("Cosido") to provide a clean, high-precision spectrum.

3.2 Spectrum Post-processing for Compliance

The system transforms raw **Power Spectral Density (PSD)** data into regulatory findings. It uses the **FCME (Forward Consecutive Mean Excision)** algorithm to estimate the noise floor dynamically. Emissions are then segmented and cross-referenced with **DANE municipal databases** to verify compliance with licensed frequency, bandwidth, and power limits.

4 Metrological Validation

The platform has undergone extensive statistical characterization ($N = 10,000$ samples) including:

- **DANL Characterization:** Using **ANOVA** testing to ensure uniformity across different hardware units.
- **IQ Balance Optimization:** Implementing **Gram-Schmidt** orthogonalization to improve the **Spurious-Free Dynamic Range (SFDR)** and suppress spectral images.
- **Frequency Accuracy:** Correcting **LO drift** using **GSM FCCH tones** as a reference to calculate parts-per-million (PPM) error.

5 Recommendations for Future Improvements

1. **Enable Automated Calibration:** Current repository analysis shows that `**pre-campaign calibration calls**` (`kal_sync`) are present but often commented out in the orchestrator; these should be fully integrated into the production state machine.
2. **Implement C-Engine Unit Testing:** While the Python code has `**168` passing tests`**` via `pytest`, the C-based RF engine currently lacks a native automated test suite.
3. **External Clock Integration:** Utilizing a `**GPSDO` (GPS Disciplined Oscillator)`**` via the HackRF's CLKIN would eliminate thermal drift and improve PPM accuracy beyond standard TCXO performance.
4. **Dynamic Resource Control:** Implementing a controller to adjust FFT size or decimation factors based on `**CPU load**` would prevent packet loss during high-bandwidth captures on the Raspberry Pi.